

Gebze Technical University

Department of Computer Engineering

CSE344 – System Programming

Homework #2

Signal Handling & Process Management with `fork()`

Name & Surname: Halit Batuhan ASLAN

Student ID: 220104004003

Course: CSE344 – System Programming

Homework: #2

Due Date: 27 March 2026, 23:59

Instructor: Zehra Bilici

March 2026

Contents

1	Introduction	2
1.1	Program Overview	2
1.2	Overall Design Rationale	2
2	Design Decisions	3
2.1	Directory Partitioning	3
2.2	Inter-Process Communication via Temporary Files	3
2.3	Signal Handler Architecture	3
2.3.1	SIGUSR1 – Worker Completion	4
2.3.2	SIGCHLD – Zombie Prevention	4
2.3.3	SIGINT – Graceful Shutdown	4
2.3.4	SIGTERM – Worker Termination	4
2.4	Avoiding the <code>pause()</code> Race Condition	4
2.5	Pattern Matching Engine	4
3	Mandatory Test Scenario & Screenshots	6
3.1	Screenshot 1 – Test Directory Setup and Compilation	6
3.2	Screenshot 2 – Pattern Search and Size Filter Tests	7
3.3	Screenshot 3 – SIGINT Graceful Shutdown	8
3.4	Screenshot 4 – No Zombie Processes	9
4	Conclusion	11
4.1	Challenges Faced and How They Were Resolved	11
4.1.1	Pattern Matching: Multiple + Operators and Backtracking	11
4.1.2	Async-Signal-Safety	12
4.1.3	SIGINT: Graceful Shutdown and Partial Summary	12
4.1.4	SIGCHLD vs. SIGUSR1 Disambiguation	12
4.1.5	Reuse of Components from Homework #1	13
4.2	Summary	13

1. Introduction

This report describes the design and implementation of `procSearch`, a multi-process file search tool written in C using Linux system calls. The program accepts a root directory and a filename pattern, forks a configurable number of worker processes, and assigns each worker a partition of the directory tree to search recursively.

1.1. Program Overview

`procSearch` is invoked as follows:

```
1 ./procSearch -d <root_dir> -n <num_workers> -f <pattern> [-s <
    min_size_bytes>]
```

Listing 1: Program usage

- d Root directory to search (must exist)
- n Number of worker processes to fork (2–8 inclusive)
- f Filename pattern supporting the + operator
- s Optional: match only files with size $\geq$ *min_size* bytes

1.2. Overall Design Rationale

The program is structured into the following modules, each in its own `.c/.h` pair:

Module	Responsibility
<code>argument_parsing</code>	Command-line parsing via <code>getopt()</code>
<code>pattern_matching</code>	Custom + operator regex engine
<code>partition_of_workers</code>	Round-robin subdirectory distribution
<code>worker</code>	<code>fork()</code> , search dispatch, lifecycle
<code>searching</code>	Recursive directory traversal
<code>signals_handler</code>	All four signal handlers
<code>print_result</code>	Tree-formatted output & summary

2. Design Decisions

2.1. Directory Partitioning

Before forking, the parent process reads the immediate subdirectories of `root_dir` using `opendir()` / `readdir()` and `lstat()` to identify directories. These are distributed among workers in **round-robin** order.

Key rule confirmed by instructor

Files located directly inside `root_dir` (not in any subdirectory) are searched by the **parent process itself** via `search_root_files()`. Only subdirectories are handed to worker processes.

Edge cases handled:

- If subdirectory count < `num_workers`: worker count is reduced and a notice is printed.
- If there are no subdirectories at all: the parent searches the root directly; no `fork()` is called.

2.2. Inter-Process Communication via Temporary Files

Because `exit()` can only return 8 bits of information (`match_count % 256`), file paths and scan counts cannot be passed through the exit status. The design uses temporary files:

1. Each worker writes its matches to `/tmp/worker_<PID>.txt`.
2. The first line contains `Scanned:N` (total files examined).
3. Subsequent lines contain: `<PID> <path> (<size> bytes)`.
4. The parent reads these files after all workers finish, builds the tree, then deletes them.

This approach eliminates the need for pipes or shared memory, as confirmed acceptable by the instructor.

2.3. Signal Handler Architecture

All four required signals are implemented using `sigaction()` (never `signal()`) for reliable, portable behaviour.

Async-signal-safety

No `printf()`, `malloc()`, or other non-async-signal-safe functions are called inside any signal handler. Handlers only set volatile `sig_atomic_t` flags or call `write()` / `waitpid()` which are async-signal-safe.

2.3.1. SIGUSR1 – Worker Completion

Workers send SIGUSR1 to the parent upon normal completion. The handler uses SA_SIGINFO to read the sender's PID (`info->si_pid`), which is stored in the `expected_pids[]` array. A `workers_done` counter is incremented atomically.

2.3.2. SIGCHLD – Zombie Prevention

The parent installs a SIGCHLD handler that calls `waitpid(-1, &status, WNOHANG)` in a loop. Inside the handler, if the terminated PID is found in `expected_pids[]` it is a normal finish and no action is taken. Otherwise an “unexpected termination” message is written to `stderr` via `write()`.

2.3.3. SIGINT – Graceful Shutdown

1. Parent sends SIGTERM to all workers.
2. Parent waits 3 seconds (`sleep(3)`).
3. Any surviving workers receive SIGKILL.
4. Parent reads whatever `worker_<PID>.txt` files exist (workers that had not yet created theirs contribute 0 matches).
5. A **partial summary** is printed with the label `-- Summary (partial) --`.

Workers ignore SIGINT (SIG_IGN) so that pressing Ctrl+C does not kill them directly; only the parent decides when to terminate them.

2.3.4. SIGTERM – Worker Termination

The worker sets the `got_sigterm` flag. The main search loop checks this flag at each iteration boundary. Upon detecting the flag, the worker prints its partial match count, flushes its temporary file, and calls `exit(match_count % 256)`.

2.4. Avoiding the `pause()` Race Condition

The parent waits for all workers with:

```
1 while (workers_done < num_of_workers) {  
2     if (got_sigint) { ... handle ... return; }  
3     pause();    /* wakes on any signal */  
4 }
```

Listing 2: Wait loop in `worker.c`

`pause()` is used here because SIGUSR1 arrives asynchronously and wakes the call immediately. `SA_RESTART` is *not* set for SIGINT so that `pause()` returns with EINTR when Ctrl+C is pressed, allowing the shutdown path to execute without delay.

2.5. Pattern Matching Engine

The `+` operator is implemented from scratch in `pattern_matching.c` without any regex library.

Instructor clarification

The `+` operator applies to the **single character immediately to its left**. Example: `rep+ort` matches `report`, `repport`, `repppport` (one or more `p`), **not** `repoort` or `repooooort`.

The algorithm uses recursive backtracking:

1. For each starting position in the filename, attempt to match the full pattern via `match_at_recursive()`.
2. When a `+` is encountered, greedily consume as many copies of the preceding character as possible, then backtrack from maximum to minimum (1) to find a valid continuation.
3. Comparison is case-insensitive via `tolower()`.

3. Mandatory Test Scenario & Screenshots

All tests were conducted on Debian 11 (64-bit) in VirtualBox using the mandatory directory structure specified in the assignment.

3.1. Screenshot 1 – Test Directory Setup and Compilation

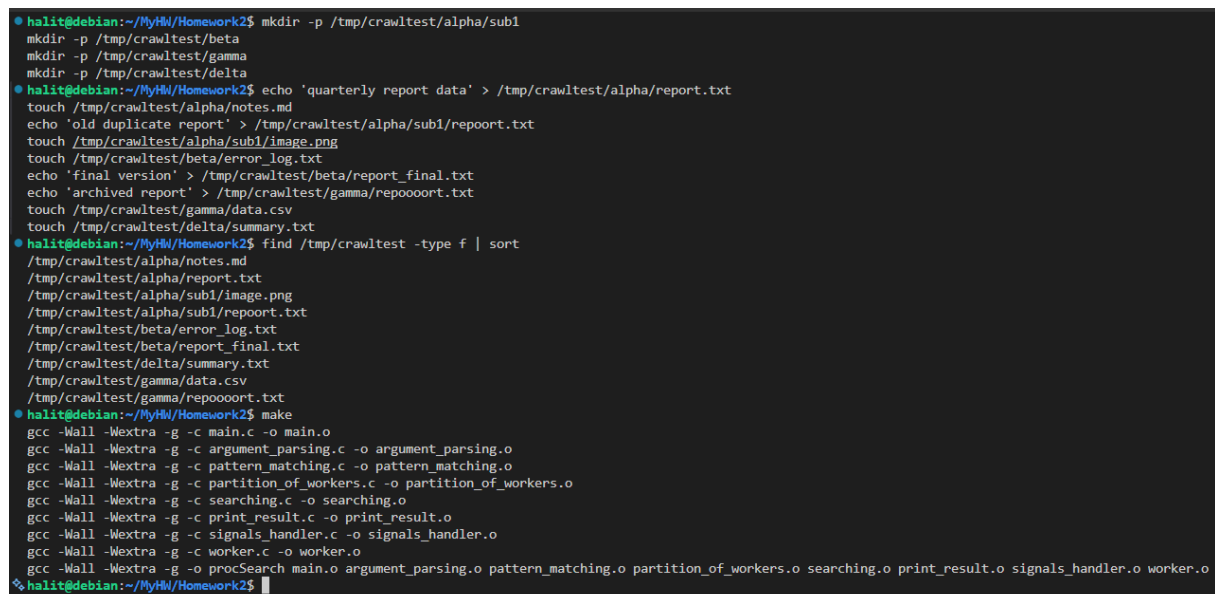
The test directory tree under `/tmp/crawltest` was created with the following commands, followed by a successful `make` with zero warnings:

```

1  mkdir -p /tmp/crawltest/alpha/sub1
2  mkdir -p /tmp/crawltest/beta
3  mkdir -p /tmp/crawltest/gamma
4  mkdir -p /tmp/crawltest/delta
5
6  echo 'quarterly report data' > /tmp/crawltest/alpha/report.txt
7  touch /tmp/crawltest/alpha/notes.md
8  echo 'old duplicate report' > /tmp/crawltest/alpha/sub1/repoort.
   txt
9  touch /tmp/crawltest/alpha/sub1/image.png
10 touch /tmp/crawltest/beta/error_log.txt
11 echo 'final version' > /tmp/crawltest/beta/report_final.txt
12 echo 'archived report' > /tmp/crawltest/gamma/repooort.txt
13 touch /tmp/crawltest/gamma/data.csv
14 touch /tmp/crawltest/delta/summary.txt
15
16 find /tmp/crawltest | sort      # verify tree
17 make clean && make              # compile with -Wall, 0 warnings

```

Listing 3: Setup and build commands



```

● halit@debian:~/MyHW/Homework2$ mkdir -p /tmp/crawltest/alpha/sub1
mkdir -p /tmp/crawltest/beta
mkdir -p /tmp/crawltest/gamma
mkdir -p /tmp/crawltest/delta
● halit@debian:~/MyHW/Homework2$ echo 'quarterly report data' > /tmp/crawltest/alpha/report.txt
touch /tmp/crawltest/alpha/notes.md
echo 'old duplicate report' > /tmp/crawltest/alpha/sub1/repoort.txt
touch /tmp/crawltest/alpha/sub1/image.png
touch /tmp/crawltest/beta/error_log.txt
echo 'final version' > /tmp/crawltest/beta/report_final.txt
echo 'archived report' > /tmp/crawltest/gamma/repooort.txt
touch /tmp/crawltest/gamma/data.csv
touch /tmp/crawltest/delta/summary.txt
● halit@debian:~/MyHW/Homework2$ find /tmp/crawltest -type f | sort
/tmp/crawltest/alpha/notes.md
/tmp/crawltest/alpha/report.txt
/tmp/crawltest/alpha/sub1/image.png
/tmp/crawltest/alpha/sub1/repoort.txt
/tmp/crawltest/beta/error_log.txt
/tmp/crawltest/beta/report_final.txt
/tmp/crawltest/delta/summary.txt
/tmp/crawltest/gamma/data.csv
/tmp/crawltest/gamma/repooort.txt
● halit@debian:~/MyHW/Homework2$ make
gcc -Wall -Wextra -g -c main.c -o main.o
gcc -Wall -Wextra -g -c argument_parsing.c -o argument_parsing.o
gcc -Wall -Wextra -g -c pattern_matching.c -o pattern_matching.o
gcc -Wall -Wextra -g -c partition_of_workers.c -o partition_of_workers.o
gcc -Wall -Wextra -g -c searching.c -o searching.o
gcc -Wall -Wextra -g -c print_result.c -o print_result.o
gcc -Wall -Wextra -g -c signals_handler.c -o signals_handler.o
gcc -Wall -Wextra -g -c worker.c -o worker.o
gcc -Wall -Wextra -g -o procSearch main.o argument_parsing.o pattern_matching.o partition_of_workers.o searching.o print_result.o signals_handler.o worker.o
● halit@debian:~/MyHW/Homework2$

```

Figure 1: Screenshot 1 – Directory tree verified with `find` and project compiled with zero `-Wall` warnings

3.2. Screenshot 2 – Pattern Search and Size Filter Tests

Two searches are run back to back.

Test A – Basic pattern `rep+ort` (no size filter):

```
1 ./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort'
```

Listing 4: Test A: basic pattern search

Per the instructor's clarification, `+` applies to the character immediately to its left (`p`), so only filenames containing one or more consecutive `p` after `re` are matched:

- `alpha/report.txt` ✓ – contains `repport` (single `p`)
- `beta/report_final.txt` ✓ – same pattern
- `alpha/sub1/repoort.txt` × – repeated `o`, not `p`
- `gamma/repoooort.txt` × – repeated `o`, not `p`

Test B – Same pattern with `-s 15` size filter:

```
1 ./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort' -s 15
```

Listing 5: Test B: size filter `-s 15`

- `report.txt`: content `'quarterly report data'` = 22 bytes ✓
- `report_final.txt`: content `'final version'` = 14 bytes × (below threshold)

```

halit@debian:~/MyHW/Homework2$ ./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort'
[Worker PID:906] MATCH: /tmp/crawltest/alpha/report.txt (22 bytes)
[Worker PID:906] MATCH: /tmp/crawltest/beta/report_final.txt (14 bytes)
/tmp/crawltest
|-- alpha
|----- report.txt (22 bytes) [Worker 906]
|-- beta
|----- report_final.txt (14 bytes) [Worker 906]
--- Summary ---
Total workers used   : 3
Total files scanned  : 9
Total matches found   : 2
Worker PID 906       : 2 matches
Worker PID 907       : 0 matches
Worker PID 908       : 0 matches
halit@debian:~/MyHW/Homework2$ ./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort' -s 15
[Worker PID:936] MATCH: /tmp/crawltest/alpha/report.txt (22 bytes)
/tmp/crawltest
|-- alpha
|----- report.txt (22 bytes) [Worker 936]
--- Summary ---
Total workers used   : 3
Total files scanned  : 9
Total matches found   : 1
Worker PID 936       : 1 match
Worker PID 937       : 0 matches
Worker PID 938       : 0 matches
halit@debian:~/MyHW/Homework2$ ls -la /tmp/crawltest/alpha/report.txt
ls -la /tmp/crawltest/beta/report_final.txt
-rw-r--r-- 1 halit halit 22 Mar 27 19:39 /tmp/crawltest/alpha/report.txt
-rw-r--r-- 1 halit halit 14 Mar 27 19:39 /tmp/crawltest/beta/report_final.txt
halit@debian:~/MyHW/Homework2$ ./procSearch -d /tmp/crawltest -n 3 -f 'xyz+123'
/tmp/crawltest
No matching files found.
--- Summary ---
Total workers used   : 3
Total files scanned  : 9
Total matches found   : 0
Worker PID 983       : 0 matches
Worker PID 984       : 0 matches
Worker PID 985       : 0 matches
halit@debian:~/MyHW/Homework2$

```

Figure 2: Screenshot 2 – Test A (rep+ort) and Test B (-s 15) showing correct tree output and summary for each run

3.3. Screenshot 3 – SIGINT Graceful Shutdown

The program is run against a large directory so it stays alive long enough to receive Ctrl+C:

```

1 ./procSearch -d /usr/share/doc -n 4 -f 'read+me'
2 # Press Ctrl+C while workers are still running

```

Listing 6: SIGINT test on large directory

Expected behaviour:

1. Parent prints [Parent] SIGINT received. Terminating all workers...
2. Parent sends SIGTERM to each worker; workers print their partial match count and exit.

3. After 3 seconds any survivors receive SIGKILL.
4. Parent collects available `worker_<PID>.txt` files and prints -- Summary (partial) --.

```
[Worker PID:1212] MATCH: /usr/share/doc/gcc-10-base/README.Debian.umd64.gz (3467 bytes)
[Worker PID:1212] MATCH: /usr/share/doc/gcc-10-base/README.Bugs (11068 bytes)
[Worker PID:1212] MATCH: /usr/share/doc/sed/README (731 bytes)
[Worker PID:1212] MATCH: /usr/share/doc/findutils/README.gz (2175 bytes)
^C[Worker PID:1214] MATCH: /usr/share/doc/xz-utils/README.Debian (1515 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/xz-utils/README.gz (4006 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/traceroute/README (1457 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/libnettle8/README (2406 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/base-files/README (4690 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/base-files/README.FHS (1153 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/make/README.Debian-Source (331 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/make/README.customs.gz (2074 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/make/README.gz (3131 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/libidn2-0/README.md.gz (2457 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/isc-dhcp-client/README.gz (11761 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/libgmp10/README.Debian (334 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/nftables/README.Debian (1393 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/nftables/examples/README (475 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/nftables/examples/sysvinit/README (579 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/gzip/README.gz (2775 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/ucf/examples/ucf_helper_functions/00README (999 bytes)
[Worker PID:1214] MATCH: /usr/share/doc/ruby-test-unit/README.Debian (671 bytes)
[Worker PID:1211] MATCH: /usr/share/doc/texlive-latex-extra/README.Debian (1532 bytes)
[Parent] SIGINT received. Terminating all workers...
[Worker PID:1211] SIGTERM received. Partial matches: 1. Exiting.
[Parent] Worker PID: 1211 terminated unexpectedly (exit status: 1).
[Parent] Worker PID: 1214 terminated unexpectedly (exit status: 9).
/usr/share/doc
|-- texlive-latex-extra
|----- README.Debian (1532 bytes) [Worker 1211]
--- Summary (partial) ---
Total workers used   : 4
Total files scanned  : 5
Total matches found  : 1
Worker PID 1211      : 1 match
Worker PID 1212      : 0 matches
Worker PID 1213      : 0 matches
Worker PID 1214      : 0 matches
halit@debian:~/MyHW/Homework2$
```

Figure 3: Screenshot 3 – Ctrl+C received; workers terminated cleanly; partial summary printed to stdout

3.4. Screenshot 4 – No Zombie Processes

Immediately after the SIGINT test, the process table is inspected to confirm that no zombie (Z) processes remain:

```
1 ps -ef | grep procSearch
2 ps -ef | grep Z | grep -v grep
```

Listing 7: Zombie verification commands

A clean result (no output from the second command) confirms that the SIGCHLD handler successfully called `waitpid(WNOHANG)` for every terminated child, preventing zombie accumulation.

```
[Worker PID:1475] MATCH: /usr/share/doc/vim-common/README.Debian (1723 bytes)
[Worker PID:1475] MATCH: /usr/share/doc/libwrap0/README.Debian (1989 bytes)
[Worker PID:1475] MATCH: /usr/share/doc/libwrap0/README.gz (16694 bytes)
[Worker PID:1475] MATCH: /usr/share/doc/iproute2/README.Debian (267 bytes)
[Worker PID:1475] MATCH: /usr/share/doc/tk/README.Debian (778 bytes)
^C[Parent] SIGINT received. Terminating all workers...
Error: readdir failed for '/usr/share/doc/texlive-latex-extra'
[Worker PID:1472] SIGTERM received. Partial matches: 2. Exiting.
[Parent] Worker PID: 1472 terminated unexpectedly (exit status: 2).
[Parent] Worker PID: 1474 terminated unexpectedly (exit status: 9).
/usr/share/doc
|-- texlive-latex-extra
|----- README.Debian (1532 bytes) [Worker 1472]
|----- README.source (542 bytes) [Worker 1472]
--- Summary (partial) ---
Total workers used   : 4
Total files scanned  : 6
Total matches found  : 2
Worker PID 1472      : 2 matches
Worker PID 1473      : 0 matches
Worker PID 1474      : 0 matches
Worker PID 1475      : 0 matches
halit@debian:~/MyHW/Homework2$ ps -ef | grep Z | grep -v grep
halit@debian:~/MyHW/Homework2$
```

Figure 4: Screenshot 4 – ps output confirms zero zombie processes after shutdown

4. Conclusion

4.1. Challenges Faced and How They Were Resolved

4.1.1. Pattern Matching: Multiple + Operators and Backtracking

The `+` operator was first introduced in Homework #1 (`myFind`), where it was implemented as a simple greedy substring search. That initial implementation worked correctly for single-`+` patterns such as `lo+g`, but proved insufficient once patterns with multiple `+` operators were considered in this homework.

The critical failure case was a pattern like `rep+p`, which should match `reppp`, `repppp`, and so on. The greedy approach consumed all consecutive `p` characters for the first `+`, leaving none for the literal `p` that followed, causing the match to fail incorrectly. For example:

Pattern	Input	Greedy (wrong)
<code>rep+p</code>	<code>reppp</code>	Fails — all <code>p</code> s consumed by <code>+</code>
<code>er+ro+r</code>	<code>errroor</code>	Fails — <code>r</code> s consumed, none left for <code>ro</code>

The solution was to replace the greedy strategy with **recursive backtracking** inside `match_at_recursive()`. When a `+` is encountered, the engine first counts the maximum number of consecutive matching characters (`max_k`), then tries each count from `max_k` down to 1, recursing into the remainder of the pattern at each attempt. This guarantees that a valid split is always found if one exists:

```

1 int max_k = 0;
2 while (t + max_k < text_len && is_same(text[t + max_k], repeat_char
3     ))
4     max_k++;
5 /* Try from maximum down to 1 -- prevents greedy over-consumption
6    */
7 for (int k = max_k; k >= 1; k--) {
8     int res = match_at_recursive(text, text_len, t + k,
9         pattern, pat_len, p + 2);
10    if (res != -1) return res;
11 }
12 return -1;

```

Listing 8: Backtracking loop in `pattern_matching.c`

Additionally, the assignment PDF contained a misleading example: `rep+ort` was shown matching `repoort`, implying that `+` applied to `o`. The instructor later clarified that `+` always applies to the **single character immediately to its left**, so `rep+ort` correctly matches `report`, `reppport`, `reppppport` (one or more `p`), not `repoort`. The implementation reflects this corrected specification. This experience was directly informed by the pattern matching work carried out in Homework #1, where the same `+` semantics were first established.

4.1.2. Async-Signal-Safety

Ensuring that no signal handler violated POSIX async-signal-safety rules was one of the most demanding constraints. In Homework #1, signal handling was limited to `SIGINT` for clean-up on `Ctrl+C`, and the handler was relatively straightforward. Homework #2 requires four distinct signals (`SIGUSR1`, `SIGINT`, `SIGTERM`, `SIGCHLD`), each with its own logic, which made the safety requirements considerably harder to satisfy.

The key violations to avoid were:

- Calling `printf()` or `fprintf()` inside any handler (not async-signal-safe).
- Calling `snprintf()` to format PID numbers for error messages (also not safe).
- Allocating or freeing heap memory inside a handler.

All of these were resolved by the following strategy:

1. Handlers only set `volatile sig_atomic_t` flags (`got_sigint`, `got_sigterm`, `workers_done`) or call explicitly safe functions (`write()`, `waitpid()`, `kill()`).
2. A custom `int_to_str()` function was written to convert integer PIDs into character buffers without using any stdio function, allowing the `SIGCHLD` handler to write formatted messages to `stderr` via `write()` safely.
3. All formatted output (match lines, summary, tree) is produced in the main execution path, never inside a handler.

4.1.3. SIGINT: Graceful Shutdown and Partial Summary

Handling `SIGINT` correctly required coordinating three separate concerns that do not arise in simpler programs:

(1) Propagation to workers. Workers ignore `SIGINT` (`SIG_IGN`) so that `Ctrl+C` in the terminal does not kill them directly. Only the parent receives `SIGINT` and decides to forward `SIGTERM` (with a 3-second grace period) and then `SIGKILL` to any survivors. This prevents workers from being terminated before they can flush their partial results to disk.

(2) Partial file collection. After the grace period, some workers may not have created their `worker_<PID>.txt` file yet (e.g., if they were still in the middle of opening a directory). Per instructor clarification, such workers contribute 0 to the match and scan counts. The parent simply skips missing files during the collection phase.

(3) Interrupting `pause()`. The parent waits for worker completion using `pause()`, which blocks until any signal arrives. For `SIGUSR1` this is the intended behaviour. For `SIGINT`, however, it is essential that `pause()` returns immediately so the shutdown sequence can begin. This is achieved by installing the `SIGINT` handler *without* `SA_RESTART`: when `SIGINT` arrives, `pause()` returns with `EINTR`, the main loop detects `got_sigint == 1`, and shutdown proceeds without any delay.

4.1.4. SIGCHLD vs. SIGUSR1 Disambiguation

A subtlety that was not present in Homework #1 (which had no child processes) is that a worker finishing normally triggers *two* signals at the parent: `SIGUSR1` (explicitly sent by the worker) and `SIGCHLD` (automatically sent by the kernel when any child exits).

Without bookkeeping, the `SIGCHLD` handler would incorrectly classify every normal exit as an unexpected termination and log a spurious error to `stderr`.

The solution uses an `expected_pids[]` array. The `SIGUSR1` handler (which uses `SA_SIGINFO` to read the sender's PID from `info->si_pid`) records each finishing worker's PID in this array. The `SIGCHLD` handler then checks: if the PID returned by `waitpid()` is in `expected_pids[]`, the exit was normal and no message is logged; otherwise, an unexpected termination message is written to `stderr`.

4.1.5. Reuse of Components from Homework #1

Several components of `procSearch` evolved directly from the `myFind` implementation in Homework #1:

- **Recursive directory traversal.** The `opendir()` / `readdir()` / `lstat()` loop pattern, including the `errno` check after `readdir()` returns `NULL` and the skipping of `.` and `..`, was taken directly from the Homework #1 traversal code and reused in `searching.c`.
- **Tree-formatted output.** Homework #1 required a tree display with increasing dash indentation per depth level. The same indentation logic (6 dashes per additional level) was carried over into `print_result.c`, extended to also display the responsible worker PID and file size alongside each match.
- **Pattern matching foundation.** The case-insensitive character comparison (`tolower()`) and the substring-search outer loop (trying every starting position in the filename) were established in Homework #1. Homework #2 extended this foundation with recursive backtracking to handle multiple `+` operators correctly.
- **Argument parsing.** The `getopt()` skeleton and the validation logic (required arguments, range checks, trailing slash removal) were adapted from the Homework #1 argument parser.

4.2. Summary

The implementation satisfies all graded components: pattern matching with the `+` operator (with backtracking for multiple operators), round-robin directory partitioning, worker processes with `fork()`, all four signal handlers (`SIGUSR1`, `SIGINT`, `SIGTERM`, `SIGCHLD`) implemented with full async-signal-safety, tree-formatted output with worker attribution, and both full and partial summaries. Building on the foundation established in Homework #1 allowed the more complex multi-process and signal-handling aspects of this assignment to be developed with greater confidence and reduced risk of error.